

1. Creational Patterns (5)

These deal with object creation mechanisms.

1. **Factory Method** – Define an interface for creating an object, but let subclasses decide which class to instantiate.
2. **Abstract Factory** – Provide an interface to create families of related/dependent objects without specifying concrete classes.
3. **Builder** – Separate the construction of a complex object from its representation, allowing the same construction process to create different representations.
4. **Prototype** – Create new objects by copying an existing object (the prototype).
5. **Singleton** – Ensure a class has only one instance and provide a global point of access.

2. Structural Patterns (7)

These deal with object composition, forming larger structures.

6. **Adapter (Wrapper)** – Convert the interface of a class into another interface clients expect.
7. **Bridge** – Decouple an abstraction from its implementation so they can vary independently.
8. **Composite** – Compose objects into tree structures to represent part-whole hierarchies.
9. **Decorator** – Attach additional responsibilities to an object dynamically.
10. **Facade** – Provide a simplified interface to a complex subsystem.
11. **Flyweight** – Use sharing to support large numbers of fine-grained objects efficiently.
12. **Proxy** – Provide a surrogate or placeholder to control access to another object.

3. Behavioral Patterns (11)

These deal with communication between objects.

13. **Chain of Responsibility** – Pass a request along a chain of handlers until one handles it.
14. **Command** – Encapsulate a request as an object, letting you parameterize clients, queue requests, and support undo.
15. **Interpreter** – Define a representation for a grammar and an interpreter to interpret sentences in the language.
16. **Iterator** – Provide a way to access elements of an aggregate object sequentially without exposing its representation.

17. **Mediator** – Define an object that encapsulates how a set of objects interact, reducing dependencies.
18. **Memento** – Capture and externalize an object's internal state so it can be restored later.
19. **Observer (Publish/Subscribe)** – Define a one-to-many dependency so that when one object changes state, dependents are notified automatically.
20. **State** – Allow an object to alter its behavior when its internal state changes.
21. **Strategy** – Define a family of algorithms, encapsulate each one, and make them interchangeable.
22. **Template Method** – Define the skeleton of an algorithm, deferring some steps to subclasses.
23. **Visitor** – Represent an operation to be performed on elements of an object structure without changing the classes of the elements.